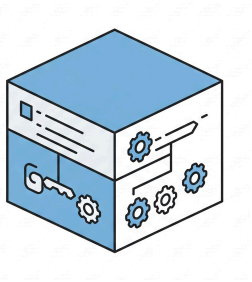
B 3.1.4 Classes and instantiation Python



B3.1.4 Construct code to define classes and instantiate objects.

This section covers defining classes, creating objects, and the role of constructors. Constructors initialize an object's state by setting initial attribute values.

Defining Classes and Instantiating Objects (Python)

This section explores the core concepts of Object-Oriented Programming (OOP) in Python. Our focus will be on defining classes, creating objects, and understanding the vital role of the **constructor** (known as the `__init__` method in Python) in initialising objects.

In OOP, we aim to model real-world entities within our programs. To achieve this, we use **classes** as blueprints or templates. A class defines the characteristics (data, called attributes in Python) and behaviours (functions, called methods in Python) that objects of that class will possess. Think of a class as a mould – it defines the form, but you can use it to create many individual items (objects) from that mould.

Defining a Class in Python

In Python, you define a class using the `class` keyword, followed by the name you want to give your class. Class names typically follow the CapitalizedWords convention. The body of the class, which is indented, contains the definitions of its attributes (variables) and methods (functions).

```
class Dog:
    # Attributes (data) of a Dog will be initialised in the constructor

    # Methods (behaviours) of a Dog
    def bark(self):
        print("Woof!")

    def eat(self):
        print("The dog is eating.")
```

In this example, we have defined a class named `Dog`. The attributes are not explicitly defined here, but will be within a special method called the constructor. We also have two methods: `bark()` and `eat()`, which define actions a `Dog` object can perform. Note the `self` parameter in the method definitions – this is a convention in Python that refers to the instance of the object itself.

Creating Objects (Instantiation)

Once you have defined a class, you can create individual instances of that class. These instances are called **objects**. Creating an object from a class is known as **instantiation**. In Python, you instantiate an object by simply calling the class name followed by parentheses. This implicitly calls the class's **constructor** method, `__init__` (if defined).

```
# Creating two Dog objects
my_dog = Dog()
another_dog = Dog()
```

Here, we are creating two `Dog` objects. The line `my_dog = Dog()` creates a new `Dog` object in memory and assigns a reference to it to the variable `my_dog`. Similarly, `another_dog = Dog()` creates a second, distinct `Dog` object. Each object will have its own set of attributes and can perform the methods defined in the `Dog` class.

The Role of the Constructor (`__init__` method)

In Python, the **constructor** is a special method named `__init__` (pronounced "dunder init"). It is automatically called when an object of the class is created. Its primary role is to **initialise the object's state** by setting initial values to its attributes. The `__init__` method always takes `self` as its first parameter, which refers to the instance of the object being created. During instantiation, you can also define additional parameters to pass initial values for the object's attributes.

Defining the `__init__` Method in Python

Here's an example of the `Dog` class with an `__init__` method that allows us to set the `breed` and `name` of a `Dog` object when it is created:

```
class Dog:
    def __init__(self, initial_breed, initial_name):
        self.breed = initial_breed
        self.name = initial_name
        self.age = 0  # Setting a default initial age
        print(f"A new Dog object named {self.name} (a {self.breed}) has been created.")

    def bark(self):
        print(f"Woof! My name is {self.name}.")

    def eat(self):
        print(f"{self.name} is eating.")

# Creating Dog objects using the constructor
my_first_dog = Dog("Labrador", "Buddy")
my_second_dog = Dog("Poodle", "Bella")

my_first_dog.bark()  # Output: Woof! My name is Buddy.
my_second_dog.eat()  # Output: Bella is eating.
```

In this updated `Dog` class:

- We have defined the constructor method: `def __init__(self, initial_breed, initial_name):`. It takes `self` as the first parameter, followed by `initial_breed` and `initial_name`.
- Inside `__init__`, we use `self.breed = initial_breed` to assign the value passed as `initial_breed` to the `breed` attribute of the object. Similarly for `name`. We also set a default `age` of 0.
- The lines `my_first_dog = Dog("Labrador", "Buddy")` and `my_second_dog = Dog("Poodle", "Bella")` demonstrate how to create `Dog` objects, providing the initial breed and name as arguments, which are then used by the `__init__` method to initialise the object's attributes.

The `__init__` method is essential for setting up the initial state of your objects when they are created. It ensures that your objects are properly configured with the necessary data to function correctly. Understanding how to define classes and instantiate objects with the `__init__` method is a fundamental aspect of Object-Oriented Programming in Python.

Try it out

Try instantiating more dogs and calling their behaviours.


Code
Run
Share
Remix
trinket
Python3
main.py
Choose File
No file chosen